FastAPI API 명세

1. 문서 목적

본 문서는 `nanji_app/Python/fastapi` 기준 주요 API를 정리한 문서이다.

대상 범위:

- 사용자 앱(iOS)에서 사용하는 API
- 관리자 웹에서 사용하는 API
- 개발/디버그용 API

기준 베이스 URL:

```
http://127.0.0.1:8000/api/v1
```

---

2. 공통 구조

서비스 구조는 다음과 같다.

```
사용자 앱 / 관리자 웹 -> FastAPI -> AWS RDS MySQL
```

앱과 웹은 DB에 직접 접근하지 않으며, 모든 데이터 조회 및 저장은 FastAPI를 통해 수행된다.

---

3. 사용자 앱용 API

3-1. 소셜 로그인

- Method: `POST`
- Path: `/auth/social-login`
- 기능: 소셜 로그인 사용자 생성 또는 갱신

요청 예시:

```
{
  "provider": "google",
  "provider_user_id": "1234567890",
  "email": "user@example.com",
  "name": "홍길동"
}
```

응답 예시:

```
{
  "user_id": 1,
  "provider": "google",
  "provider_user_id": "1234567890",
  "email": "user@example.com",
  "name": "홍길동",
  "is_new_user": true
}
```

비고:

- 로그인 성공 시 `user` 테이블을 갱신한다.
- `user_action_log`에 `login` 로그를 기록한다.

3-2. 주차장 목록 조회

- Method: `GET`
- Path: `/parking/lots`
- 기능: 전체 주차장 목록 조회

비고:

- 앱 홈, 대체 주차장 화면, 관리자 웹에서 사용한다.

3-3. 현재 주차 현황 조회

- Method: `GET`
- Path: `/parking/current/{parking_lot_id}`
- 기능: 특정 주차장의 최신 실시간 상태 조회

응답 주요 필드:

- `parking_lot_name`
- `has_data`
- `item.ps_available_spaces`
- `item.ps_occupied_spaces`
- `item.ps_occupancy_rate`
- `item.ps_congestion_level`

비고:

- 최신 `parking_status_log` 기준으로 반환한다.
- 최신 row가 존재하는 경우 시간 경과 여부와 관계없이 반환한다.

3-4. 주차 현황 이력 조회

- Method: `GET`

- Path: `/parking/history/{parking_lot_id}`
- 기능: 특정 주차장의 과거 상태 로그 조회

비고:

- 차트 및 이력성 분석에 사용한다.

3-5. 예측 데이터 조회

- Method: `GET`
- Path: `/predictions/{parking_lot_id}`
- 기능: 특정 주차장의 예측 데이터 목록 조회

쿼리 파라미터:

- `limit`
- `target_date` (선택)

응답 주요 필드:

- `pp_predicted_time`
- `pp_predicted_occupied_spaces`
- `pp_predicted_available_spaces`
- `pp_predicted_occupancy_rate`
- `pp_predicted_congestion_level`
- `pp_model_version`

사용 화면:

- 앱 홈 미래 예측
- 출발 타이밍
- 시간대별 분석
- 관리자 웹 예측 분석
- 앱 홈 미래 예측
- 출발 타이밍
- 시간대별 분석
- 관리자 웹 예측 분석

---

4. 사용자 설정/행동 로그 API

4-1. 내 프로필 조회

- Method: `GET`
- Path: `/me/profile`

- 기능: 로그인 사용자 프로필 조회

헤더:

- `X-User-ID`

4-2. 즐겨찾기 목록 조회

- Method: `GET`
- Path: `/me/favorites`
- 기능: 사용자 즐겨찾기 목록 조회

헤더:

- `X-User-ID`

4-3. 즐겨찾기 토글

- Method: `POST`
- Path: `/me/favorites/toggle`
- 기능: 즐겨찾기 추가/제거

헤더:

- `X-User-ID`

요청 예시:

```
{
  "parking_lot_id": 1
}
```

비고:

- 즐겨찾기 추가/제거 후 `favorite_add`, `favorite_remove` 행동 로그를 기록할 수 있다.

4-4. 알림 설정 목록 조회

- Method: `GET`
- Path: `/me/notifications`
- 기능: 사용자 알림 설정 목록 조회

헤더:

- `X-User-ID`

4-5. 알림 설정 저장

- Method: `POST`
- Path: `/me/notifications`
- 기능: 알림 on/off 및 설정 저장

헤더:

- `X-User-ID`

요청 예시:

```
{
  "parking_lot_id": 1,
  "notification_type": "parking_status",
  "is_enabled": true
}
```

4-6. 사용자 행동 로그 저장

- Method: `POST`
- Path: `/me/actions`
- 기능: 앱에서 발생한 사용자 행동 로그 저장

헤더:

- `X-User-ID`

요청 예시:

```
{
  "action_type": "prediction_view",
  "parking_lot_id": 1,
  "action_target": "home_prediction_card",
  "action_value": "view",
  "source_page": "home",
  "session_id": "session-uuid"
}
```

허용되는 `action_type` 예시:

- `login`
- `congestion_view`
- `prediction_view`
- `departure_timing_view`
- `map_view`
- `favorite_add`
- `favorite_remove`
- `notification_set`

---

5. 관리자 웹용 API

5-1. 관리자 로그인

- Method: `POST`
- Path: `/admin/auth/login`
- 기능: 관리자 웹 로그인

요청 예시:

```
{
  "admin_id": "admin",
  "password": "1111"
}
```

응답 예시:

```
{
  "admin_id": 1,
  "admin_login_id": "admin",
  "admin_name": "관리자",
  "admin_role": "admin"
}
```

비고:

- `admin.a_email` 전체 또는 `@` 앞부분을 로그인 ID로 사용할 수 있다.
- `a_password`는 해시 저장을 기준으로 한다.

5-2. 관리자 대시보드 요약

- Method: `GET`
- Path: `/admin/dashboard/overview`
- 기능: 관리자 메인 대시보드 로딩

주요 데이터:

- 오늘 방문자
- 혼잡도 조회
- 예측 조회
- 출발 타이밍
- 지도 클릭
- 즐겨찾기
- 주차장 현황
- 운영 인사이트

- 시간대별 사용
- 기능별 사용
- 퍼널

5-3. 주차장 분석 요약

- Method: `GET`
- Path: `/admin/parking-analysis/overview`
- 기능: 주차장 분석 화면 로딩

주요 데이터:

- 주차장별 현재 상태
- 지도 사용 비율
- 즐겨찾기 순위
- 요일별 선택 추이

5-4. 예측 분석 요약

- Method: `GET`
- Path: `/admin/prediction-analysis/overview`
- 기능: 예측 분석 화면 로딩

주요 데이터:

- 시간대별 예측 분포
- 예측 정확도 경향
- 예측 후 사용자 행동
- 혼잡도 예측 비교

5-5. 사용자 행동 요약

- Method: `GET`
- Path: `/admin/user-behavior/overview`
- 기능: 사용자 행동 분석 화면 로딩

주요 데이터:

- 전체 사용자
- 오늘 활성 사용자
- 세션 시간 분포
- 재방문 패턴
- 기능별 빈도
- 이탈 구간

5-6. 실시간 활동 로그 요약

- Method: `GET`
- Path: `/admin/activity-log/overview`
- 기능: 최근 사용자 활동 로그 조회

비고:

- 최근 `user_action_log` 30건 기준
- 전체 누적 로그 수
- 최근 활동 종류별 개수
- 사용자별 활동 내용

---

6. 개발/디버그용 API

6-1. DB 상태 확인

- Method: `GET`
- Path: `/db/status`

6-2. 테이블 목록

- Method: `GET`
- Path: `/db/tables`

6-3. 특정 테이블 컬럼 조회

- Method: `GET`
- Path: `/db/tables/{table_name}/columns`

비고:

- `ENABLE_DEBUG_DB_API=true` 인 경우에만 접근 가능
- 운영 환경에서는 기본 비활성화 상태를 사용한다

---

7. 공통 보조 API

7-1. Ping

- Method: `GET`
- Path: `/ping`
- 기능: 라우터 동작 확인

7-2. Health

- Method: `GET`
- Path: `/health`
- 기능: FastAPI 서버 기동 여부 확인

---

8. 정리

현재 API는 다음 세 범주로 구분된다.

1. 사용자 앱용 API
2. 관리자 웹용 API
3. 개발/디버그용 API

구성은 다음과 같다.

- 사용자 앱: `auth`, `parking`, `prediction`, `user_pref`
- 관리자 웹: `admin/*`
- DB 직접 접근 없음, 모든 데이터는 FastAPI를 통해 RDS에 저장 및 조회

앱과 웹은 서로 다른 클라이언트이며, 데이터 기준은 하나의 FastAPI와 하나의 AWS RDS를 공유하는 구조이다.